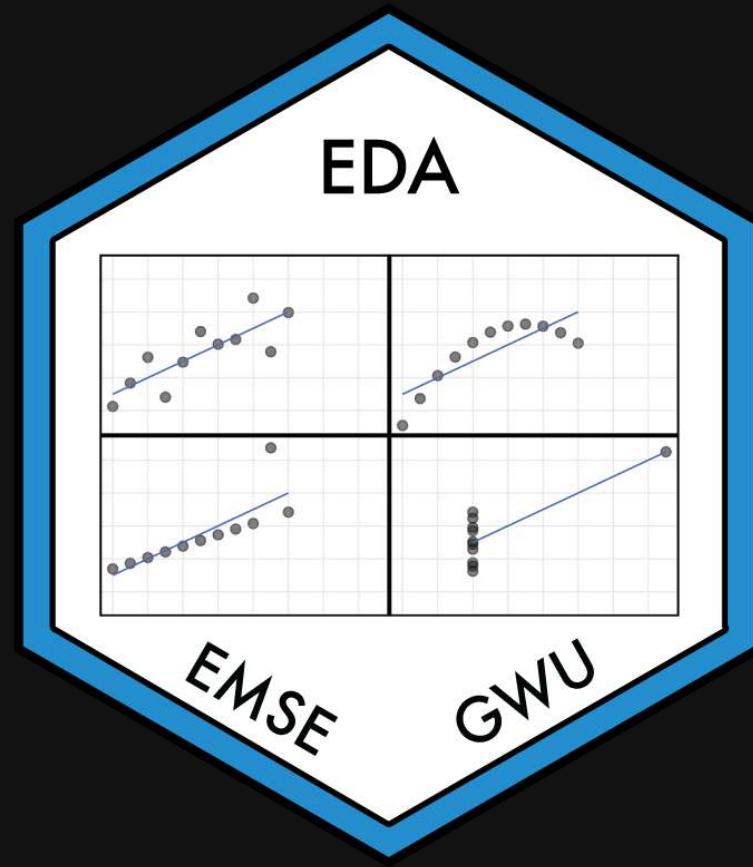




Joins & Data Cleaning

🏛️ EMSE 4572/6572: Exploratory Data Analysis

👤 John Paul Helveston

📅 September 23, 2026

Tip of the week

Copy-paste magic with datapasta

Useful for “small data”: e.g., U.S. State Abbreviations

Today's data

"Clean" data

```
1 wildlife_impacts <- read_csv(file.path('data', 'wildlife_impacts.csv'))
2 milk_production <- read_csv(file.path('data', 'milk_production.csv'))
3 msleep <- read_csv(file.path('data', 'msleep.csv'))
```

"Messy" data

```
1 wind <- read_excel(file.path('data', 'US_State_Wind_Energy_Facts_2018.xlsx'))
2 hot_dogs <- read_excel(file.path('data', 'hot_dog_winners.xlsx'))
```

Plus two new packages:

```
1 # For manipulating dates
2 install.packages('lubridate')
3
4 # For cleaning column names
5 install.packages('janitor')
```

Week 5: *Joins & Data Cleaning*

1. Merging datasets with joins
2. Are your variables the right *type*?
3. Are your variables the right *name*?
4. Re-coding variables
5. Dates
6. Dealing with messy Excel files

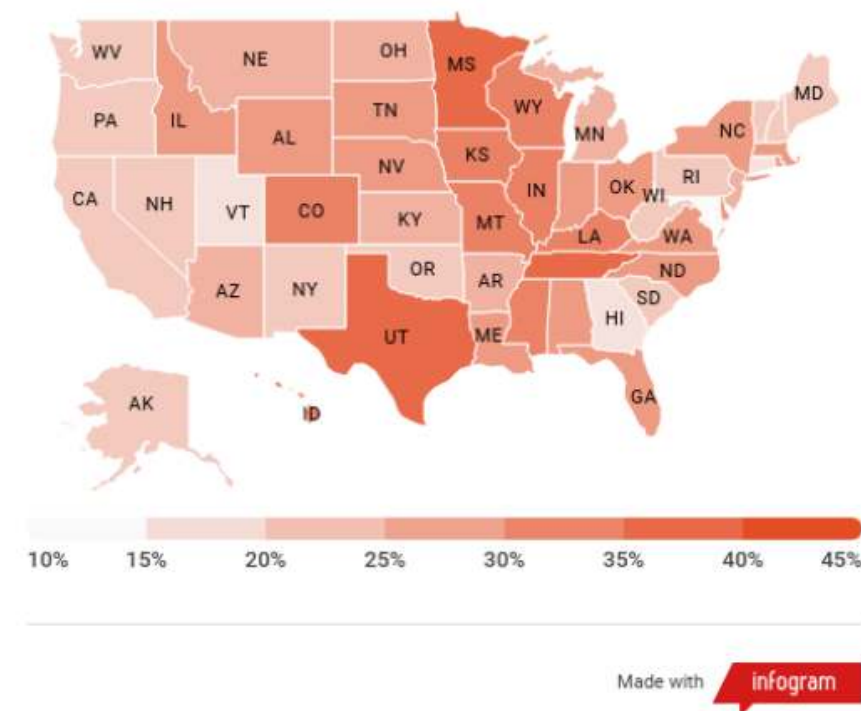
Week 5: *Joins & Data Cleaning*

1. Merging datasets with joins
2. Are your variables the right *type*?
3. Are your variables the right *name*?
4. Re-coding variables
5. Dates
6. Dealing with messy Excel files

What's wrong with this map?

A state breakdown of who's skipping medications because they're too costly

Across the U.S., 28% of consumers ages 19 to 64 say they have not taken their prescription drugs as their health care provider has prescribed them because of cost, [according to AARP research](#). Here's a look at the percentage by state of residents who say they stopped taking medication due to cost.



Likely culprit: Merging two columns

```
1 head(names)
```

```
#>   state_name
#> 1   Alabama
#> 2    Alaska
#> 3   Arizona
#> 4  Arkansas
#> 5 California
#> 6   Colorado
```

```
1 head(abbs)
```

```
#>   state_abb
#> 1        AK
#> 2        AL
#> 3        AR
#> 4        AZ
```

```
1 result <- bind_cols(names, abbs)
2 head(result)
```

```
#>   state_name state_abb
#> 1   Alabama        AK
#> 2    Alaska        AL
#> 3   Arizona        AR
#> 4  Arkansas        AZ
#> 5 California       CA
#> 6   Colorado       CO
```

Joins

1. `inner_join()`
2. `left_join()` / `right_join()`
3. `full_join()`

Example: `band_members` & `band_instruments`

```
1 band_members
```

```
#> # A tibble: 3 × 2
#>   name  band
#>   <chr> <chr>
#> 1 Mick  Stones
#> 2 John  Beatles
#> 3 Paul  Beatles
```

```
1 band_instruments
```

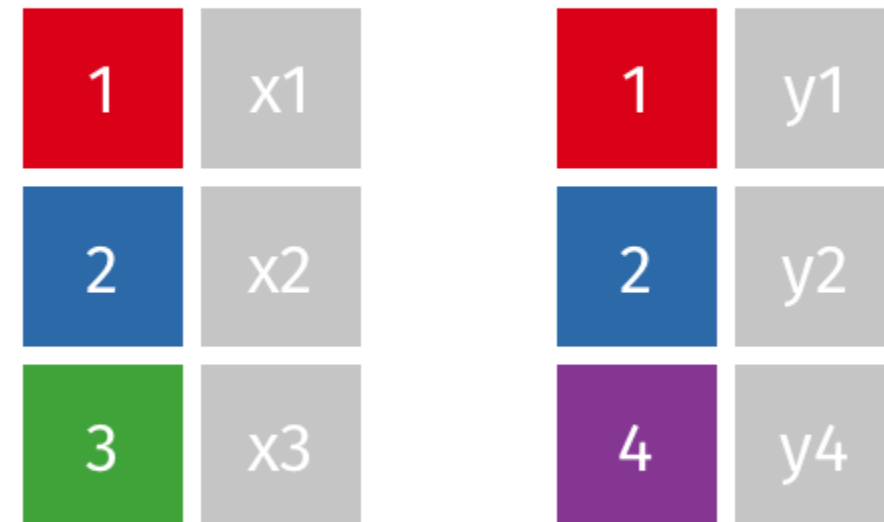
```
#> # A tibble: 3 × 2
#>   name  plays
#>   <chr> <chr>
#> 1 John  guitar
#> 2 Paul  bass
#> 3 Keith guitar
```

inner_join()

```
1 band_members %>%  
2   inner_join(band_instruments)
```

```
#> # A tibble: 2 × 3  
#>   name  band  plays  
#>   <chr> <chr> <chr>  
#> 1 John  Beatles guitar  
#> 2 Paul  Beatles  bass
```

inner_join(x, y)



full_join()

```
1 band_members %>%  
2   full_join(band_instruments)
```

```
#> # A tibble: 4 × 3  
#>   name  band  plays  
#>   <chr> <chr> <chr>  
#> 1 Mick  Stones <NA>  
#> 2 John  Beatles guitar  
#> 3 Paul  Beatles bass  
#> 4 Keith <NA>   guitar
```

full_join(x, y)

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

left_join()

```
1 band_members %>%  
2   left_join(band_instruments)
```

```
#> # A tibble: 3 × 3  
#>   name  band  plays  
#>   <chr> <chr> <chr>  
#> 1 Mick  Stones <NA>  
#> 2 John  Beatles guitar  
#> 3 Paul  Beatles bass
```

left_join(x, y)

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

right_join()

```
1 band_members %>%  
2   right_join(band_instruments)
```

```
#> # A tibble: 3 × 3  
#>   name  band  plays  
#>   <chr> <chr> <chr>  
#> 1 John  Beatles guitar  
#> 2 Paul  Beatles bass  
#> 3 Keith <NA>   guitar
```

right_join(x, y)

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

Specify the joining variable name

```
1 band_members %>%  
2   left_join(band_instruments)
```

```
#> Joining with `by = join_by(name)`
```

```
#> # A tibble: 3 × 3  
#>   name  band  plays  
#>   <chr> <chr>  <chr>  
#> 1 Mick  Stones <NA>  
#> 2 John  Beatles guitar  
#> 3 Paul  Beatles bass
```

```
1 band_members %>%  
2   left_join(  
3     band_instruments,  
4     by = 'name'  
5   )
```

```
#> # A tibble: 3 × 3  
#>   name  band  plays  
#>   <chr> <chr>  <chr>  
#> 1 Mick  Stones <NA>  
#> 2 John  Beatles guitar  
#> 3 Paul  Beatles bass
```

Specify the joining variable name

If the names differ, use `by = c("left_name" = "joining_name")`

```
1 band_members
```

```
#> # A tibble: 3 × 2  
#>   name band  
#>   <chr> <chr>  
#> 1 Mick  Stones  
#> 2 John  Beatles  
#> 3 Paul  Beatles
```

```
1 band_instruments2
```

```
#> # A tibble: 3 × 2  
#>   artist plays  
#>   <chr>   <chr>  
#> 1 John   guitar
```

```
1 band_members %>%  
2   left_join(  
3     band_instruments2,  
4     by = c("name" = "artist")  
5   )
```

```
#> # A tibble: 3 × 3  
#>   name band plays  
#>   <chr> <chr> <chr>  
#> 1 Mick  Stones <NA>  
#> 2 John  Beatles guitar  
#> 3 Paul  Beatles bass
```

Specify the joining variable name

Or just rename the joining variable in a pipe

```
1 band_members
```

```
#> # A tibble: 3 × 2  
#>   name band  
#>   <chr> <chr>  
#> 1 Mick  Stones  
#> 2 John  Beatles  
#> 3 Paul  Beatles
```

```
1 band_instruments2
```

```
#> # A tibble: 3 × 2  
#>   artist plays  
#>   <chr>   <chr>  
#> 1 John   guitar
```

```
1 band_members %>%  
2   rename(artist = name) %>%  
3   left_join(  
4     band_instruments2,  
5     by = "artist"  
6   )
```

```
#> # A tibble: 3 × 3  
#>   artist band   plays  
#>   <chr>   <chr>   <chr>  
#> 1 Mick   Stones  <NA>  
#> 2 John   Beatles guitar  
#> 3 Paul   Beatles  bass
```

Your turn

15:00

1. Create a data frame called `state_data` by joining the data frames `states_abbs` and `milk_production` and then selecting the variables `region`, `state_name`, `state_abb`. **Hint:** Use the `distinct()` function to drop repeated rows.

Your result should look like this:

```
1 head(state_data)
```

```
#> # A tibble: 6 × 3
#>   region    state_name state_abb
#>   <chr>    <chr>      <chr>
#> 1 Northeast Maine       ME
#> 2 Northeast New Hampshire NH
#> 3 Northeast Vermont      VT
#> 4 Northeast Massachusetts MA
#> 5 Northeast Rhode Island RI
#> 6 Northeast Connecticut  CT
```

2. Join the `state_data` data frame to the `wildlife_impacts` data frame, adding the variables `region` and `state_name`

```
1 glimpse(wildlife_impacts)
```

```
#> Rows: 56,978
#> Columns: 24
#> $ region      <chr> "Northeast", "Northeast", "Northeast", "Northeast", "Northeast",
#> $ state_name   <chr> "Maine", "Maine", "Maine", "Maine", "Maine", "Maine", "Maine", "M
#> $ state_abb    <chr> "ME", "ME", "ME", "ME", "ME", "ME", "ME", "ME", "ME", "ME", "ME",
#> $ incident_date <dtm> 2018-10-23, 2018-10-07, 2018-10-05, 2018-10-05, 2017-07-25, 2016
#> $ airport_id   <chr> "KPWM", "KPWM", "KPWM", "KPWM", "KPWM", "KPWM", "KPWM", "KPWM", "
#> $ airport      <chr> "PORTLAND INTL JETPORT (ME)", "PORTLAND INTL JETPORT (ME)", "PORT
#> $ operator     <chr> "AMERICAN AIRLINES", "AMERICAN AIRLINES", "AMERICAN AIRLINES", "A
#> $ atype        <chr> "A-320", "A-319", "A-319", "EMB-190", "EMB-170", "A-319", "A-319"
#> $ type_eng     <chr> "D", "D", "D", "D", "D", "D", "D", "D", "D", "C", "D", "D", "D",
#> $ species_id   <chr> "UNKBS", "ZX302", "ZS010", "I1102", "K3310", "YH004", "UNKB", "YL
#> $ species      <chr> "Unknown bird - small", "Swamp sparrow", "Blackpoll warbler", "Gr
#> $ damage       <chr> "N", NA, "N", "M?", "N", "N", "N", "N", "N", "N", NA, "N", "N", N
#> $ num_engs     <dbl> 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
#> $ incident_month <dbl> 10, 10, 10, 10, 7, 11, 11, 10, 7, 8, 11, 7, 5, 4, 4, 5, 10, 9, 8,
#> $ incident_year <dbl> 2018, 2018, 2018, 2018, 2017, 2016, 2016, 2016, 2016, 1990, 2018,
#> $ time_of_day   <chr> NA, "Night", "Night", "Day", "Dawn", "Day", "Day", "Day", "Day",
#> $ time         <dbl> 1310, 1035, 2200, 1645, 645, 1345, 1346, 1400, 1100, NA, 610, 191
#> $ height       <dbl> 15, NA, 1000, 0, 0, 0, 0, NA, NA, 2000, 0, 50, 0, 0, NA, 0, 0, 0,
#> $ speed        <dbl> 150, NA, 140, 110, NA, NA, NA, NA, NA, NA, 250, 100, NA, 150, NA, NA,
#> $ phase_of_flt <chr> "departure", "arrival", "arrival", "arrival", "arrival", "arrival",
#> $ sky          <chr> "Overcast", "Some Cloud", "Some Cloud", "Some Cloud", "Some Cloud
#> $ precip       <chr> "None", "None", "None", "None", "None", "None", "None", NA, "Rain", "None
#> $ cost_repairs_infl_adj <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
#> $ weekday_name <ord> Tue, Sun, Fri, Fri, Tue, Mon, Mon, Sat, Sat, Wed, Tue, Sun, Sat,
```

Week 5: *Joins & Data Cleaning*

1. Merging datasets with joins
2. *Are your variables the right type?*
3. Are your variables the right *name*?
4. Re-coding variables
5. Dates
6. Dealing with messy Excel files

Always check variable types after reading in data!

```
1 wind <- read_excel(file.path(  
2   'data',  
3   'US_State_Wind_Energy_Facts_2018.xlsx'  
4 ))  
5  
6 glimpse(wind)
```

```
#> Rows: 50  
#> Columns: 7  
#> $ Ranking <chr> "1.0", "2.0", "3.0", "4.0", "5.0", "6.0"  
#> $ State <chr> "TEXAS", "OKLAHOMA", "IOWA", "CALIFORNIA"  
#> $ `Installed Capacity (MW)` <dbl> 23262, 7495, 7312, 5686, 5110, 4464, 3000  
#> $ `Equivalent Homes Powered` <chr> "6235000.0", "2268000.0", "1935000.0",  
#> $ `Total Investment ($ Millions)` <chr> "42000.0", "13700.0", "14200.0", "12600.0"  
#> $ `Wind Projects Online` <dbl> 136, 45, 107, 104, 35, 49, 98, 31, 25,  
#> $ `# of Wind Turbines` <chr> "12750.0", "3717.0", "4145.0", "6972.0"
```

Be careful converting strings to numbers!

as.numeric()

```
1 as.numeric(c("2.1", "3.7", "4.50"))
```

```
#> [1] 2.1 3.7 4.5
```

```
1 as.numeric(c("$2.1", "$3.7", "$4.50"))
```

```
#> [1] NA NA NA
```

parse_number()

```
1 parse_number(c("2.1", "3.7", "4.50"))
```

```
#> [1] 2.1 3.7 4.5
```

```
1 parse_number(c("$2.1", "$3.7", "$4.50"))
```

```
#> [1] 2.1 3.7 4.5
```

```
1 parse_number(c("1-800-123-4567"))
```

```
#> [1] 1
```

Week 5: *Joins & Data Cleaning*

1. Merging datasets with joins
2. Are your variables the right *type*?
3. *Are your variables the right name?*
4. Re-coding variables
5. Dates
6. Dealing with messy Excel files

Renaming made easy

```
janitor::clean_names()
```



```
1 wind <- read_excel(file.path(  
2   'data',  
3   'US_State_Wind_Energy_Facts_2018.xlsx'  
4 ))  
5  
6 glimpse(wind)
```

```
#> Rows: 50  
#> Columns: 7  
#> $ Ranking <chr> "1.0", "  
#> $ State <chr> "TEXAS",  
#> $ `Installed Capacity (MW)` <dbl> 23262, 7  
#> $ `Equivalent Homes Powered` <chr> "6235000  
#> $ `Total Investment ($ Millions)` <chr> "42000.0  
#> $ `Wind Projects Online` <dbl> 136, 45,  
#> $ `# of Wind Turbines` <chr> "12750.0
```

Renaming made easy

`janitor::clean_names()`



```
1 library(janitor)
2
3 wind <- read_excel(file.path(
4   'data',
5   'US_State_Wind_Energy_Facts_2018.xlsx'
6 )) %>%
7   clean_names()
8
9 glimpse(wind)
```

```
#> Rows: 50
#> Columns: 7
#> $ ranking      <chr> "1.0", "2.0",
#> $ state        <chr> "TEXAS", "OKLA
#> $ installed_capacity_mw <dbl> 23262, 7495, 7
#> $ equivalent_homes_powered <chr> "6235000.0", "
#> $ total_investment_millions <chr> "42000.0", "13
```

Renaming made easy

`janitor::clean_names()`



```
1 library(janitor)
2
3 wind <- read_excel(file.path(
4   'data',
5   'US_State_Wind_Energy_Facts_2018.xlsx'
6 )) %>%
7   clean_names(case = 'lower_camel')
8
9 glimpse(wind)
```

```
#> Rows: 50
#> Columns: 7
#> $ ranking      <chr> "1.0", "2.0", "3
#> $ state        <chr> "TEXAS", "OKLAHO
#> $ installedCapacityMw <dbl> 23262, 7495, 731
#> $ equivalentHomesPowered <chr> "6235000.0", "22
#> $ totalInvestmentMillions <chr> "42000.0", "1370
```

Renaming made easy

`janitor::clean_names()`



```
1 library(janitor)
2
3 wind <- read_excel(file.path(
4   'data',
5   'US_State_Wind_Energy_Facts_2018.xlsx'
6 )) %>%
7   clean_names(case = 'screaming_snake')
8
9 glimpse(wind)
```

```
#> Rows: 50
#> Columns: 7
#> $ RANKING      <chr> "1.0", "2.0",
#> $ STATE        <chr> "TEXAS", "OKLA
#> $ INSTALLED_CAPACITY_MW <dbl> 23262, 7495, 7
#> $ EQUIVALENT_HOMES_POWERED <chr> "6235000.0", "
#> $ TOTAL_INVESTMENT_MILLIONS <chr> "42000.0", "13
```

`select()`: more powerful than you probably thought

Example: data on sleeping patterns of different mammals

```
1 glimpse(msleep)
```

```
#> Rows: 83
#> Columns: 11
#> $ name      <chr> "Cheetah", "Owl monkey", "Mountain beaver",
#> $ genus     <chr> "Acinonyx", "Aotus", "Apodonta", "Blarina
#> $ vore      <chr> "carni", "omni", "herbi", "omni", "herbi",
#> $ order     <chr> "Carnivora", "Primates", "Rodentia", "Soric
#> $ conservation <chr> "lc", NA, "nt", "lc", "domesticated", NA, "
#> $ sleep_total <dbl> 12.1, 17.0, 14.4, 14.9, 4.0, 14.4, 8.7, 7.0
#> $ sleep_rem  <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.2, 1.4, NA, 2.9,
#> $ sleep_cycle <dbl> NA, NA, NA, 0.1333333, 0.6666667, 0.7666667
#> $ awake     <dbl> 11.90, 7.00, 9.60, 9.10, 20.00, 9.60, 15.30
#> $ brainwt   <dbl> NA, 0.01550, NA, 0.00029, 0.42300, NA, NA,
#> $ bodywt    <dbl> 50.000, 0.480, 1.350, 0.019, 600.000, 3.850
```

`select()`: more powerful than you probably thought

Use `select()` to choose which columns to **keep**

```
1 msleep %>%
2   select(name:order, sleep_total:sleep_cycle) %>%
3   glimpse()
```

```
#> Rows: 83
#> Columns: 7
#> $ name      <chr> "Cheetah", "Owl monkey", "Mountain beav
#> $ genus     <chr> "Acinonyx", "Aotus", "Aplodontia", "Bla
#> $ vore       <chr> "carni", "omni", "herbi", "omni", "herb
#> $ order      <chr> "Carnivora", "Primates", "Rodentia", "S
#> $ sleep_total <dbl> 12.1, 17.0, 14.4, 14.9, 4.0, 14.4, 8.7,
#> $ sleep_rem  <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.2, 1.4, NA, 2
#> $ sleep_cycle <dbl> NA, NA, NA, 0.1333333, 0.6666667, 0.766
```

Use `select()` to choose which columns to **drop**

```
1 msleep %>%
2   select(-(name:order)) %>%
3   glimpse()
```

```
#> Rows: 83
#> Columns: 7
#> $ conservation <chr> "lc", NA, "nt", "lc", "dom
#> $ sleep_total  <dbl> 12.1, 17.0, 14.4, 14.9, 4.0
#> $ sleep_rem    <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.2
#> $ sleep_cycle  <dbl> NA, NA, NA, 0.1333333, 0.6
#> $ awake        <dbl> 11.90, 7.00, 9.60, 9.10, 2
#> $ brainwt      <dbl> NA, 0.01550, NA, 0.00029,
#> $ bodywt       <dbl> 50.000, 0.480, 1.350, 0.01
```

Select columns based on **partial column names**

Select columns that start with "sleep":

```
1 msleep %>%  
2   select(name, starts_with("sleep")) %>%  
3   glimpse()
```

```
#> Rows: 83  
#> Columns: 4  
#> $ name      <chr> "Cheetah", "Owl monkey", "Mountai  
#> $ sleep_total <dbl> 12.1, 17.0, 14.4, 14.9, 4.0, 14.4  
#> $ sleep_rem   <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.2, 1.4,  
#> $ sleep_cycle <dbl> NA, NA, NA, 0.1333333, 0.6666667,
```

Select columns that contain "eep" and end with "wt":

```
1 msleep %>%  
2   select(contains("eep"), ends_with("wt")) %>%  
3   glimpse()
```

```
#> Rows: 83  
#> Columns: 5  
#> $ sleep_total <dbl> 12.1, 17.0, 14.4, 14.9, 4.0, 14.4  
#> $ sleep_rem   <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.2, 1.4,  
#> $ sleep_cycle <dbl> NA, NA, NA, 0.1333333, 0.6666667,  
#> $ brainwt     <dbl> NA, 0.01550, NA, 0.00029, 0.42300  
#> $ bodywt      <dbl> 50.000, 0.480, 1.350, 0.019, 600.
```

Select columns based on their **data type**

Select only numeric columns:

```
1 msleep %>%
2   select_if(is.numeric) %>%
3   glimpse()
```

```
#> Rows: 83
#> Columns: 6
#> $ sleep_total <dbl> 12.1, 17.0, 14.4
#> $ sleep_rem   <dbl> NA, 1.8, 2.4, 2.
#> $ sleep_cycle <dbl> NA, NA, NA, 0.13
#> $ awake       <dbl> 11.90, 7.00, 9.6
#> $ brainwt     <dbl> NA, 0.01550, NA,
#> $ bodywt      <dbl> 50.000, 0.480, 1
```

Select only character columns:

```
1 msleep %>%
2   select_if(is.character) %>%
3   glimpse()
```

```
#> Rows: 83
#> Columns: 5
#> $ name          <chr> "Cheetah", "Owl
#> $ genus         <chr> "Acinonyx", "Ao
#> $ vore          <chr> "carni", "omni"
#> $ order         <chr> "Carnivora", "P
#> $ conservation <chr> "lc", NA, "nt",
```

Use `select()` to **reorder** variables

```
1 msleep %>%
2   select(everything()) %>%
3   glimpse()
```

```
#> Rows: 83
#> Columns: 11
#> $ name      <chr> "Cheetah", "Owl monkey", "
#> $ genus     <chr> "Acinonyx", "Aotus", "Aplo
#> $ vore       <chr> "carni", "omni", "herbi",
#> $ order      <chr> "Carnivora", "Primates", "
#> $ conservation <chr> "lc", NA, "nt", "lc", "dom
#> $ sleep_total <dbl> 12.1, 17.0, 14.4, 14.9, 4.
#> $ sleep_rem  <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.
#> $ sleep_cycle <dbl> NA, NA, NA, 0.1333333, 0.6
#> $ awake      <dbl> 11.90, 7.00, 9.60, 9.10, 2
#> $ brainwt    <dbl> NA, 0.01550, NA, 0.00029,
#> $ bodywt     <dbl> 50.000, 0.480, 1.350, 0.01
```

```
1 msleep %>%
2   select(conservation, awake, everything()) %>%
3   glimpse()
```

```
#> Rows: 83
#> Columns: 11
#> $ conservation <chr> "lc", NA, "nt", "lc", "domesticated",
#> $ awake        <dbl> 11.90, 7.00, 9.60, 9.10, 20.00, 9.60,
#> $ name         <chr> "Cheetah", "Owl monkey", "Mountain bea
#> $ genus        <chr> "Acinonyx", "Aotus", "Aplodontia", "B
#> $ vore         <chr> "carni", "omni", "herbi", "omni", "he
#> $ order        <chr> "Carnivora", "Primates", "Rodentia", "
#> $ sleep_total  <dbl> 12.1, 17.0, 14.4, 14.9, 4.0, 14.4, 8.7
#> $ sleep_rem    <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.2, 1.4, NA,
#> $ sleep_cycle  <dbl> NA, NA, NA, 0.1333333, 0.6666667, 0.76
#> $ awake       <dbl> 11.90, 7.00, 9.60, 9.10, 20.00, 9.60,
#> $ brainwt      <dbl> NA, 0.01550, NA, 0.00029, 0.42300, NA,
#> $ bodywt       <dbl> 50.000, 0.480, 1.350, 0.019, 600.000,
```

Use `select()` to **rename** variables

Use `rename()` to just change the name

```
1 msleep %>%
2   rename(
3     animal = name,
4     extinction_threat = conservation
5   ) %>%
6   glimpse()
```

```
#> Rows: 83
#> Columns: 11
#> $ animal      <chr> "Cheetah", "Owl monkey", "M
#> $ genus       <chr> "Acinonyx", "Aotus", "Aplo
#> $ vore        <chr> "carni", "omni", "herbi", "
#> $ order       <chr> "Carnivora", "Primates", "R
#> $ extinction_threat <chr> "lc", NA, "nt", "lc", "dome
#> $ sleep_total  <dbl> 12.1, 17.0, 14.4, 14.9, 4.0
#> $ sleep_rem   <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.2
#> $ sleep_cycle <dbl> NA, NA, NA, 0.1333333, 0.66
#> $ awake       <dbl> 11.90, 7.00, 9.60, 9.10, 20
#> $ brainwt     <dbl> NA, 0.01550, NA, 0.00029, 0
#> $ bodywt      <dbl> 50.000, 0.480, 1.350, 0.019
```

Use `select()` to change the name **and drop everything else**

```
1 msleep %>%
2   select(
3     animal = name,
4     extinction_threat = conservation
5   ) %>%
6   glimpse()
```

```
#> Rows: 83
#> Columns: 2
#> $ animal      <chr> "Cheetah", "Owl monkey", "M
#> $ extinction_threat <chr> "lc", NA, "nt", "lc", "dome
```

Use `select()` to **rename** variables

Use `rename()` to just change the name

```
1 msleep %>%
2   rename(
3     animal = name,
4     extinction_threat = conservation
5   ) %>%
6   glimpse()
```

```
#> Rows: 83
#> Columns: 11
#> $ animal      <chr> "Cheetah", "Owl monkey", "M
#> $ genus       <chr> "Acinonyx", "Aotus", "Aplod
#> $ vore        <chr> "carni", "omni", "herbi", "
#> $ order       <chr> "Carnivora", "Primates", "R
#> $ extinction_threat <chr> "lc", NA, "nt", "lc", "dome
#> $ sleep_total  <dbl> 12.1, 17.0, 14.4, 14.9, 4.0
#> $ sleep_rem    <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.2
#> $ sleep_cycle  <dbl> NA, NA, NA, 0.1333333, 0.66
#> $ awake       <dbl> 11.90, 7.00, 9.60, 9.10, 20
#> $ brainwt     <dbl> NA, 0.01550, NA, 0.00029, 0
#> $ bodywt      <dbl> 50.000, 0.480, 1.350, 0.019
```

Use `select()` + `everything()` to change names **and keep everything else**

```
1 msleep %>%
2   select(
3     animal = name,
4     extinction_threat = conservation,
5     everything()
6   ) %>%
7   glimpse()
```

```
#> Rows: 83
#> Columns: 11
#> $ animal      <chr> "Cheetah", "Owl monkey", "M
#> $ extinction_threat <chr> "lc", NA, "nt", "lc", "dome
#> $ genus       <chr> "Acinonyx", "Aotus", "Aplod
#> $ vore        <chr> "carni", "omni", "herbi", "
#> $ order       <chr> "Carnivora", "Primates", "R
#> $ sleep_total  <dbl> 12.1, 17.0, 14.4, 14.9, 4.0
#> $ sleep_rem    <dbl> NA, 1.8, 2.4, 2.3, 0.7, 2.2
#> $ sleep_cycle  <dbl> NA, NA, NA, 0.1333333, 0.66
#> $ awake       <dbl> 11.90, 7.00, 9.60, 9.10, 20
#> $ brainwt     <dbl> NA, 0.01550, NA, 0.00029, 0
#> $ bodywt      <dbl> 50.000, 0.480, 1.350, 0.019
```

Your turn

Read in the

`hot_dog_winners.xlsx` file and
adjust the variable names and
types to the following:

```
#> Rows: 42
#> Columns: 7
#> $ year                <dbl> 1980, 1981, 1982
#> $ competitor.mens     <chr> "Paul Siederman
#> $ competitor.womens   <chr> NA, NA, NA, NA,
#> $ dogs_eaten.mens     <dbl> 9.10, 11.00, 11.
#> $ dogs_eaten.womens   <dbl> NA, NA, NA, NA,
#> $ country.mens        <chr> "United States",
#> $ country.womens      <chr> NA, NA, NA, NA,
```

	A	B	C	D	E	F	G
1	Year	Mens	Dogs eaten	Country	Womens	Dogs eaten	Country
2	1980	Paul Siederman & Joe Baldini	9.1	United States			
3	1981	Thomas DeBerry	11	United States			
4	1982	Steven Abrams	11	United States			
5	1983	Luis Llamas	19.5	Mexico			
6	1984	Birgit Felden	9.5	Germany			
7	1985	Oscar Rodriguez	11.75	United States			
8	1986	Mark Heller	15.5	United States			
9	1987	Don Wolfman	12	United States			
10	1988	Jay Green	14	United States			
11	1989	Jay Green	13	United States			
12	1990	Mike DeVito	16	United States			
13	1991	Frank Dellarosa	21.5*	United States			
14	1992	Frank Dellarosa	19	United States			
15	1993	Mike DeVito	17	United States			
16	1994	Mike DeVito	20	United States			
17	1995	Edward Krachie	19.5	United States			
18	1996	Edward Krachie	22.25*	United States			
19	1997	Hirofumi Nakajima	24.5*	Japan			
20	1998	Hirofumi Nakajima	19	Japan			
21	1999	Steve Keiner	20.25	United States			
22	2000	Kazutoyo Arai	25.13*	Japan			
23	2001	Takeru Kobayashi	50*	Japan			
24	2002	Takeru Kobayashi	50.5*	Japan			
25	2003	Takeru Kobayashi	44.5	Japan			
26	2004	Takeru Kobayashi	53.5*	Japan			
27	2005	Takeru Kobayashi	49	Japan			
28	2006	Takeru Kobayashi	53.75*	Japan			
29	2007	Joey Chestnut	66*	United States			
30	2008	Joey Chestnut	59	United States			
31	2009	Joey Chestnut	68*	United States			
32	2010	Joey Chestnut	54	United States			
33	2011	Joey Chestnut	62	United States	Sonya Thomas	40*	United States
34	2012	Joey Chestnut	68	United States	Sonya Thomas	45*	United States
35	2013	Joey Chestnut	69*	United States	Sonya Thomas	36.75	United States
36	2014	Joey Chestnut	61	United States	Miki Sudo	34	United States
37	2015	Matt Stonie	62	United States	Miki Sudo	38	United States
38	2016	Joey Chestnut	70*	United States	Miki Sudo	38.5	United States
39	2017	Joey Chestnut	72*	United States	Miki Sudo	41	United States
40	2018	Joey Chestnut	74*	United States	Miki Sudo	37	United States
41	2019	Joey Chestnut	71	United States	Miki Sudo	31	United States
42							
43	Notes: * means new record						

15:00

Break

05 : 00

Week 5: *Joins & Data Cleaning*

1. Merging datasets with joins
2. Are your variables the right *type*?
3. Are your variables the right *name*?
4. Re-coding variables
5. Dates
6. Dealing with messy Excel files

Recoding with `ifelse()`

Example: Create a variable, `cost_high`, that is `TRUE` if the repair costs were greater than the median costs and `FALSE` otherwise.

```
1 wildlife_impacts1 <- wildlife_impacts %>%
2   rename(cost = cost_repairs_infl_adj) %>%
3   filter(!is.na(cost)) %>%
4   mutate(
5     cost_median = median(cost),
6     cost_high = ifelse(cost > cost_median, TRUE, FALSE)
7   )
8
9 wildlife_impacts1 %>%
10  select(cost, cost_median, cost_high) %>%
11  head()
```

```
#> # A tibble: 6 × 3
#>   cost cost_median cost_high
#>   <dbl>      <dbl> <lgl>
#> 1  1000        26783 FALSE
#> 2   200        26783 FALSE
#> 3 10000        26783 FALSE
#> 4 100000        26783  TRUE
#> 5  20000        26783 FALSE
#> 6 487000        26783  TRUE
```

Recoding with **nested** `ifelse()`

Create a variable, `season`, based on the `incident_month` variable.

```
1 wildlife_impacts2 <- wildlife_impacts %>%
2   mutate(
3     season = ifelse(
4       incident_month %in% c(3, 4, 5),
5       'spring',
6       ifelse(
7         incident_month %in% c(6, 7, 8),
8         'summer',
9         ifelse(
10          incident_month %in% c(9, 10, 11),
11          'fall',
12          'winter'
13        )
14      )
15    )
16  )
17
18 wildlife_impacts2 %>%
19   distinct(incident_month, season) %>%
20   head()
```

```
#> # A tibble: 6 × 2
#>   incident_month season
#>       <dbl> <chr>
#> 1         12 winter
#> 2         11 fall
#> 3         10 fall
#> 4          9 fall
#> 5          8 summer
#> 6          7 summer
```

Recoding with `case_when()`

Create a variable, `season`, based on the `incident_month` variable.

Note: If you don't include the final `TRUE ~ 'winter'` condition, you'll get `NA` for those cases.

```
1 wildlife_impacts2 <- wildlife_impacts %>%
2   mutate(
3     season = case_when(
4       incident_month %in% c(3, 4, 5) ~ 'spring',
5       incident_month %in% c(6, 7, 8) ~ 'summer',
6       incident_month %in% c(9, 10, 11) ~ 'fall',
7       TRUE ~ 'winter'
8     )
9   )
10
11 wildlife_impacts2 %>%
12   distinct(incident_month, season) %>%
13   head()
```

```
#> # A tibble: 6 × 2
#>   incident_month season
#>   <dbl> <chr>
#> 1      12 winter
#> 2      11 fall
#> 3      10 fall
#> 4       9 fall
#> 5       8 summer
#> 6       7 summer
```

Recoding with `case_when()` with `between()`

Create a variable, `season`, based on the `incident_month` variable.

```
1 wildlife_impacts2 <- wildlife_impacts %>%
2   mutate(
3     season = case_when(
4       between(incident_month, 3, 5) ~ 'spring',
5       between(incident_month, 6, 8) ~ 'summer',
6       between(incident_month, 9, 11) ~ 'fall',
7       TRUE ~ 'winter'
8     )
9   )
10
11 wildlife_impacts2 %>%
12   distinct(incident_month, season) %>%
13   head()
```

```
#> # A tibble: 6 × 2
#>   incident_month season
#>   <dbl> <chr>
#> 1      12 winter
#> 2      11 fall
#> 3      10 fall
#> 4       9 fall
#> 5       8 summer
#> 6       7 summer
```

case_when() is “cleaner” than ifelse()

Convert the `num_engs` variable into a word of the number.

ifelse()

```
1 wildlife_impacts3 <- wildlife_impacts %>%
2   mutate(
3     num_engs = ifelse(
4       num_engs == 1,
5       'one',
6       ifelse(
7         num_engs == 2,
8         'two',
9         ifelse(
10          num_engs == 3,
11          'three',
12          ifelse(
13            num_engs == 4,
14            'four',
15            as.character(num_engs)
16          )
17        )
18      )
19    )
20  )
```

case_when()

```
1 wildlife_impacts3 <- wildlife_impacts %>%
2   mutate(
3     num_engs = case_when(
4       num_engs == 1 ~ 'one',
5       num_engs == 2 ~ 'two',
6       num_engs == 3 ~ 'three',
7       num_engs == 4 ~ 'four'
8     )
9   )
10
11 unique(wildlife_impacts3$num_engs)
```

```
#> [1] "two" NA "three" "four" "one"
```

Break a single variable into two with `separate()`

```
1 tb_rates
```

```
#> # A tibble: 6 × 3  
#>   country      year rate  
#>   <chr>      <dbl> <chr>  
#> 1 Afghanistan 1999 745/19987071  
#> 2 Afghanistan 2000 2666/20595360  
#> 3 Brazil      1999 37737/172006362  
#> 4 Brazil      2000 80488/174504898  
#> 5 China       1999 212258/127291527  
#> 6 China       2000 213766/128042858
```

```
1 tb_rates %>%  
2   separate(rate, into = c("cases", "population"))
```

```
#> # A tibble: 6 × 4  
#>   country      year cases population  
#>   <chr>      <dbl> <chr>    <chr>  
#> 1 Afghanistan 1999 745      19987071  
#> 2 Afghanistan 2000 2666     20595360  
#> 3 Brazil      1999 37737    172006362  
#> 4 Brazil      2000 80488    174504898  
#> 5 China       1999 212258   1272915272  
#> 6 China       2000 213766   1280428583
```

Break a single variable into two with `separate()`

```
1 tb_rates
```

```
#> # A tibble: 6 × 3  
#>   country      year rate  
#>   <chr>      <dbl> <chr>  
#> 1 Afghanistan 1999 745/19987071  
#> 2 Afghanistan 2000 2666/20595360  
#> 3 Brazil      1999 37737/172006362  
#> 4 Brazil      2000 80488/174504898  
#> 5 China       1999 212258/127291527  
#> 6 China       2000 213766/128042858
```

```
1 tb_rates %>%  
2   separate(  
3     rate,  
4     into = c("cases", "population"),  
5     sep = "/"  
6   )
```

```
#> # A tibble: 6 × 4  
#>   country      year cases  population  
#>   <chr>      <dbl> <chr>    <chr>  
#> 1 Afghanistan 1999 745      19987071  
#> 2 Afghanistan 2000 2666     20595360  
#> 3 Brazil      1999 37737    172006362  
#> 4 Brazil      2000 80488    174504898  
#> 5 China       1999 212258   1272915272  
#> 6 China       2000 213766   1280428583
```

Break a single variable into two with `separate()`

```
1 tb_rates
```

```
#> # A tibble: 6 × 3  
#>   country      year rate  
#>   <chr>      <dbl> <chr>  
#> 1 Afghanistan 1999 745/19987071  
#> 2 Afghanistan 2000 2666/20595360  
#> 3 Brazil      1999 37737/172006362  
#> 4 Brazil      2000 80488/174504898  
#> 5 China       1999 212258/127291527  
#> 6 China       2000 213766/128042858
```

```
1 tb_rates %>%  
2   separate(  
3     rate,  
4     into = c("cases", "population"),  
5     sep = "/",  
6     convert = TRUE  
7   )
```

```
#> # A tibble: 6 × 4  
#>   country      year cases population  
#>   <chr>      <dbl> <int>      <int>  
#> 1 Afghanistan 1999     745    19987071  
#> 2 Afghanistan 2000     2666    20595360  
#> 3 Brazil      1999    37737    172006362  
#> 4 Brazil      2000    80488    174504898  
#> 5 China       1999   212258    1272915272  
#> 6 China       2000   213766    1280428583
```

You can also break up a variable by an index

```
1 tb_rates
```

```
#> # A tibble: 6 × 3  
#>   country      year rate  
#>   <chr>      <dbl> <chr>  
#> 1 Afghanistan 1999 745/19987071  
#> 2 Afghanistan 2000 2666/20595360  
#> 3 Brazil      1999 37737/172006362  
#> 4 Brazil      2000 80488/174504898  
#> 5 China       1999 212258/127291527  
#> 6 China       2000 213766/128042858
```

```
1 tb_rates %>%  
2   separate(  
3     year,  
4     into = c("century", "year"),  
5     sep = 2  
6   )
```

```
#> # A tibble: 6 × 4  
#>   country      century year  rate  
#>   <chr>      <chr>   <chr> <chr>  
#> 1 Afghanistan 19      99    745/19987071  
#> 2 Afghanistan 20      00    2666/20595360  
#> 3 Brazil      19      99    37737/172006362  
#> 4 Brazil      20      00    80488/174504898  
#> 5 China       19      99    212258/1272915272  
#> 6 China       20      00    213766/1280428583
```

unite(): The opposite of separate()

```
1 tb_rates
```

```
#> # A tibble: 6 × 3  
#>   country      year rate  
#>   <chr>      <dbl> <chr>  
#> 1 Afghanistan 1999 745/19987071  
#> 2 Afghanistan 2000 2666/20595360  
#> 3 Brazil      1999 37737/172006362  
#> 4 Brazil      2000 80488/174504898  
#> 5 China       1999 212258/127291527  
#> 6 China       2000 213766/128042858
```

```
1 tb_rates %>%  
2   separate(year, into = c("century", "year"), sep = 2)  
3   unite(year_new, century, year)
```

```
#> # A tibble: 6 × 3  
#>   country      year_new rate  
#>   <chr>      <chr>      <chr>  
#> 1 Afghanistan 19_99      745/19987071  
#> 2 Afghanistan 20_00      2666/20595360  
#> 3 Brazil      19_99      37737/172006362  
#> 4 Brazil      20_00      80488/174504898  
#> 5 China       19_99      212258/1272915272  
#> 6 China       20_00      213766/1280428583
```

unite(): The opposite of separate()

```
1 tb_rates
```

```
#> # A tibble: 6 × 3  
#>   country      year rate  
#>   <chr>      <dbl> <chr>  
#> 1 Afghanistan 1999 745/19987071  
#> 2 Afghanistan 2000 2666/20595360  
#> 3 Brazil       1999 37737/172006362  
#> 4 Brazil       2000 80488/174504898  
#> 5 China        1999 212258/127291527  
#> 6 China        2000 213766/128042858
```

```
1 tb_rates %>%  
2   separate(year, into = c("century", "year"), sep = 2)  
3   unite(year_new, century, year, sep = "")
```

```
#> # A tibble: 6 × 3  
#>   country      year_new rate  
#>   <chr>      <chr>      <chr>  
#> 1 Afghanistan 1999      745/19987071  
#> 2 Afghanistan 2000      2666/20595360  
#> 3 Brazil       1999      37737/172006362  
#> 4 Brazil       2000      80488/174504898  
#> 5 China        1999      212258/1272915272  
#> 6 China        2000      213766/1280428583
```

Week 5: *Joins & Data Cleaning*

1. Merging datasets with joins
2. Are your variables the right *type*?
3. Are your variables the right *name*?
4. Re-coding variables
5. **Dates**
6. Dealing with messy Excel files



Create dates from strings - **order is the ONLY thing that matters!**

Year-Month-Day

```
1 ymd( '2026-09-09' )
```

```
#> [1] "2026-09-09"
```

Create dates from strings - **order is the ONLY thing that matters!**

Year-Month-Day

```
1 ymd( '2026-09-09' )
```

```
#> [1] "2026-09-09"
```

```
1 ymd( '2026 sep 9' )
```

```
#> [1] "2026-09-09"
```

Create dates from strings - **order is the ONLY thing that matters!**

Year-Month-Day

```
1 ymd('2026-09-09')
```

```
#> [1] "2026-09-09"
```

```
1 ymd('2026 Sep 9')
```

```
#> [1] "2026-09-09"
```

```
1 ymd('2026 Sep. 9')
```

```
#> [1] "2026-09-09"
```

```
1 ymd('2026 september 9')
```

```
#> [1] "2026-09-09"
```

Month-Day-Year

```
1 mdy('September 9, 2026')
```

```
#> [1] "2026-09-09"
```

```
1 mdy('Sep. 9, 2026')
```

```
#> [1] "2026-09-09"
```

```
1 mdy('Sep 9 2026')
```

```
#> [1] "2026-09-09"
```

Day-Month-Year

```
1 dmy('9 September 2026')
```

```
#> [1] "2026-09-09"
```

```
1 dmy('9 Sep. 2026')
```

```
#> [1] "2026-09-09"
```

```
1 dmy('9 Sep. 2026')
```

```
#> [1] "2026-09-09"
```

Check out the `lubridate` cheat sheet

Extracting information from dates

```
1 date <- today()  
2 date
```

```
#> [1] "2026-09-22"
```

```
1 # Get the year  
2 year(date)
```

```
#> [1] 2026
```

Extracting information from dates

```
1 date <- today()
2 date
```

```
#> [1] "2026-09-22"
```

```
1 # Get the year
2 year(date)
```

```
#> [1] 2026
```

```
1 # Get the month
2 month(date)
```

```
#> [1] 9
```

```
1 # Get the month name
2 month(date, label = TRUE, abbr = FALSE)
```

```
#> [1] September
#> Levels: January < February < March < April < May < June < July
```

```
1 # Get the day
2 day(date)
```

```
#> [1] 22
```

```
1 # Get the weekday
2 wday(date)
```

```
#> [1] 3
```

```
1 # Get the weekday name
2 wday(date, label = TRUE, abbr = TRUE)
```

```
#> [1] Tue
#> Levels: Sun < Mon < Tue < Wed < Thu < Fri < Sat
```

Quick practice

On what day of the week were you born?

```
1 wday("2026-09-23", label = TRUE)
```

```
#> [1] Wed
```

```
#> Levels: Sun < Mon < Tue < Wed < Thu < Fri <
```

Modifying date elements

```
1 date <- today()  
2 date
```

```
#> [1] "2026-09-22"
```

```
1 # Change the year  
2 year(date) <- 2016  
3 date
```

```
#> [1] "2016-09-22"
```

```
1 # Change the day  
2 day(date) <- 30
```

```
1 date
```

```
#> [1] "2016-09-30"
```

Quick practice

What do you think will happen if we do this?

```
1 date <- ymd("2026-02-28")  
2 day(date) <- 30
```

```
1 date
```

```
#> [1] "2026-03-02"
```

Your turn

20:00

1. Use `case_when()` to modify the `phase_of_flight` variable in the `wildlife_impacts` data:

- The values `'approach'`, `'arrival'`, `'descent'`, and `'landing roll'` should be merged into a single value called `'arrival'`.
- The values `'climb'`, `'departure'`, and `'take-off run'` should be merged into a single value called `'departure'`.
- All other values should be called `'other'`.

Before:

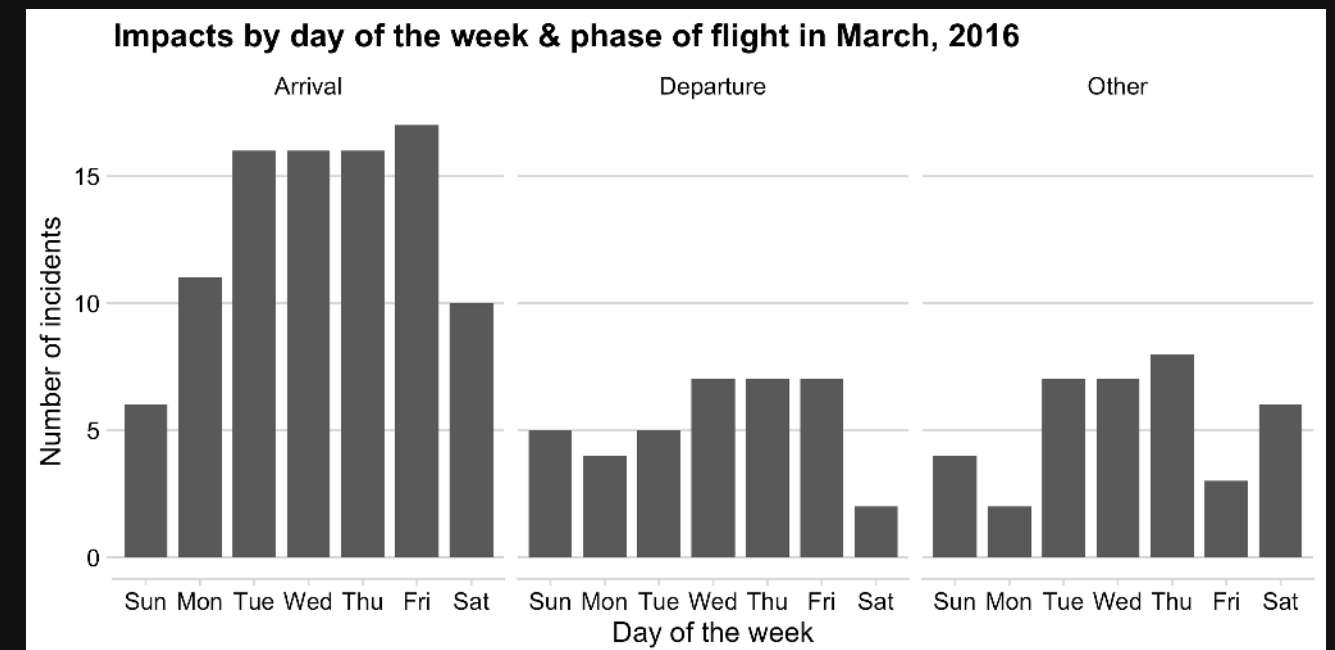
```
1 unique(str_to_lower(wildlife_impacts$phase_of_flight))
```

```
#> [1] "climb" "landing roll" NA "approach" "t
```

After:

```
#> [1] "departure" "arrival" "other"
```

2. Use the **lubridate** package to create a new variable, `weekday_name`, from the `incident_date` variable in the `wildlife_impacts` data.
3. Use `weekday_name` and `phase_of_flight` to make this plot of "arrival" and "departure" impacts from **Mar. 2016**.



Week 5: *Joins & Data Cleaning*

1. Merging datasets with joins
2. Are your variables the right *type*?
3. Are your variables the right *name*?
4. Re-coding variables
5. Dates
6. Dealing with messy Excel files

Reminders:

- You have **11** days until your [Mini Project 1](#) is due.
- Look at [HW 4](#), due next week
- Quiz 2 next week

When columns are repeated

Example: Winners of Nathan's hot dog eating contest

Stragies

1. divide & conquer

2. pivot long, separate, pivot wide

	A	B	C	D	E	F	G
1	Year	Mens	Dogs eaten	Country	Womens	Dogs eaten	Country
2	1980	Paul Siederman & Joe Baldini	9.1	United States			
3	1981	Thomas DeBerry	11	United States			
4	1982	Steven Abrams	11	United States			
5	1983	Luis Llamas	19.5	Mexico			
6	1984	Birgit Felden	9.5	Germany			
7	1985	Oscar Rodriguez	11.75	United States			
8	1986	Mark Heller	15.5	United States			
9	1987	Don Wolfman	12	United States			
10	1988	Jay Green	14	United States			
11	1989	Jay Green	13	United States			
12	1990	Mike DeVito	16	United States			
13	1991	Frank Dellarosa	21.5*	United States			
14	1992	Frank Dellarosa	19	United States			
15	1993	Mike DeVito	17	United States			
16	1994	Mike DeVito	20	United States			
17	1995	Edward Krachie	19.5	United States			
18	1996	Edward Krachie	22.25*	United States			
19	1997	Hirofumi Nakajima	24.5*	Japan			
20	1998	Hirofumi Nakajima	19	Japan			
21	1999	Steve Keiner	20.25	United States			
22	2000	Kazutoyo Arai	25.13*	Japan			
23	2001	Takeru Kobayashi	50*	Japan			
24	2002	Takeru Kobayashi	50.5*	Japan			
25	2003	Takeru Kobayashi	44.5	Japan			
26	2004	Takeru Kobayashi	53.5*	Japan			
27	2005	Takeru Kobayashi	49	Japan			
28	2006	Takeru Kobayashi	53.75*	Japan			
29	2007	Joey Chestnut	66*	United States			
30	2008	Joey Chestnut	59	United States			
31	2009	Joey Chestnut	68*	United States			
32	2010	Joey Chestnut	54	United States			
33	2011	Joey Chestnut	62	United States	Sonya Thomas	40*	United States
34	2012	Joey Chestnut	68	United States	Sonya Thomas	45*	United States
35	2013	Joey Chestnut	69*	United States	Sonya Thomas	36.75	United States
36	2014	Joey Chestnut	61	United States	Miki Sudo	34	United States
37	2015	Matt Stonie	62	United States	Miki Sudo	38	United States
38	2016	Joey Chestnut	70*	United States	Miki Sudo	38.5	United States
39	2017	Joey Chestnut	72*	United States	Miki Sudo	41	United States
40	2018	Joey Chestnut	74*	United States	Miki Sudo	37	United States
41	2019	Joey Chestnut	71	United States	Miki Sudo	31	United States
42							
43	Notes: * means new record						

Strategy 1: divide & conquer

Steps:

1. Read in the data
2. Clean the names
3. Remove * note at bottom of table

```
1 hot_dogs <- read_excel(  
2   file.path('data', 'hot_dog_winners.xlsx'),  
3   sheet = 'hot_dog_winners'  
4 ) %>%  
5   clean_names() %>%  
6   dplyr::filter(!is.na(mens))  
7  
8 glimpse(hot_dogs)
```

```
#> Rows: 40
#> Columns: 7
#> $ year      <chr> "1980", "1981", "1982", "1983", "19
#> $ mens      <chr> "Paul Siederman & Joe Baldini", "Th
#> $ dogs_eaten_3 <chr> "9.1", "11", "11", "19.5", "9.5", "
#> $ country_4  <chr> "United States", "United States", "
#> $ womens     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA,
#> $ dogs_eaten_6 <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA,
#> $ country_7  <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA,
```

Strategy 1: divide & conquer

Steps

1. Read in the data
2. Clean the names
3. Remove * note at bottom of table
4. **Split data into two competitions with the same variable names**
5. **Create new variable in each data frame: `competition`**

```
1 hot_dogs_m <- hot_dogs %>%
2   select(
3     year,
4     competitor = mens,
5     dogs_eaten = dogs_eaten_3,
6     country = country_4
7   ) %>%
8   mutate(competition = 'Mens')
9
10 hot_dogs_w <- hot_dogs %>%
11   select(
12     year,
13     competitor = womens,
14     dogs_eaten = dogs_eaten_6,
15     country = country_7
16   ) %>%
17   mutate(competition = 'Womens') %>%
18   dplyr::filter(!is.na(competitor))
```

Strategy 1: divide & conquer

Steps

1. Read in the data
2. Clean the names
3. Remove * note at bottom of table
4. Split data into two competitions with the same variable names
5. Create new variable in each data frame: `competition`
6. **Merge data together with `bind_rows()`**
7. **Clean up final data frame**

```
1 hot_dogs <- bind_rows(hot_dogs_m, hot_dogs_w) %>%
2   mutate(
3     new_record = str_detect(dogs_eaten, "\\*"),
4     dogs_eaten = parse_number(dogs_eaten),
5     year = as.numeric(year)
6   )
7
8 glimpse(hot_dogs)
```

```
#> Rows: 49
#> Columns: 6
#> $ year      <dbl> 1980, 1981, 1982, 1983, 1984, 1985,
#> $ competitor <chr> "Paul Siederman & Joe Baldini", "Tho
#> $ dogs_eaten <dbl> 9.10, 11.00, 11.00, 19.50, 9.50, 11.
#> $ country    <chr> "United States", "United States", "U
#> $ competition <chr> "Mens", "Mens", "Mens", "Mens", "Mer
#> $ new_record  <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, F
```

	A	B	C	D	E	F	G
1	Year	Mens	Dogs eaten	Country	Womens	Dogs eaten	Country
2	1980	Paul Siederman & Joe Baldini	9.1	United States			
3	1981	Thomas DeBerry	11	United States			
4	1982	Steven Abrams	11	United States			
5	1983	Luis Llamas	19.5	Mexico			
6	1984	Birgit Felden	9.5	Germany			
7	1985	Oscar Rodriguez	11.75	United States			
8	1986	Mark Heller	15.5	United States			
9	1987	Don Wolfman	12	United States			
10	1988	Jay Green	14	United States			
11	1989	Jay Green	13	United States			
12	1990	Mike DeVito	16	United States			
13	1991	Frank Dellarosa	21.5*	United States			
14	1992	Frank Dellarosa	19	United States			
15	1993	Mike DeVito	17	United States			
16	1994	Mike DeVito	20	United States			
17	1995	Edward Krachie	19.5	United States			
18	1996	Edward Krachie	22.25*	United States			
19	1997	Hirofumi Nakajima	24.5*	Japan			
20	1998	Hirofumi Nakajima	19	Japan			
21	1999	Steve Keiner	20.25	United States			
22	2000	Kazutoyo Arai	25.13*	Japan			
23	2001	Takeru Kobayashi	50*	Japan			
24	2002	Takeru Kobayashi	50.5*	Japan			
25	2003	Takeru Kobayashi	44.5	Japan			
26	2004	Takeru Kobayashi	53.5*	Japan			
27	2005	Takeru Kobayashi	49	Japan			
28	2006	Takeru Kobayashi	53.75*	Japan			
29	2007	Joey Chestnut	66*	United States			
30	2008	Joey Chestnut	59	United States			
31	2009	Joey Chestnut	68*	United States			
32	2010	Joey Chestnut	54	United States			
33	2011	Joey Chestnut	62	United States	Sonya Thomas	40*	United States
34	2012	Joey Chestnut	68	United States	Sonya Thomas	45*	United States
35	2013	Joey Chestnut	69*	United States	Sonya Thomas	36.75	United States
36	2014	Joey Chestnut	61	United States	Miki Sudo	34	United States
37	2015	Matt Stonie	62	United States	Miki Sudo	38	United States
38	2016	Joey Chestnut	70*	United States	Miki Sudo	38.5	United States
39	2017	Joey Chestnut	72*	United States	Miki Sudo	41	United States
40	2018	Joey Chestnut	74*	United States	Miki Sudo	37	United States
41	2019	Joey Chestnut	71	United States	Miki Sudo	31	United States
42							
43	Notes: * means new record						

```
1 head(hot_dogs)
```

```
#> # A tibble: 6 × 6
#>   year competitor dogs_eaten country competition new_recor
#>   <dbl> <chr>      <dbl> <chr>      <chr>      <lgl>
#> 1  1980 Paul Siederman & Joe Baldini    9.1 United States Mens      FALSE
#> 2  1981 Thomas DeBerry      11 United States Mens      FALSE
#> 3  1982 Steven Abrams      11 United States Mens      FALSE
#> 4  1983 Luis Llamas     19.5 Mexico      Mens      FALSE
#> 5  1984 Birgit Felden      9.5 Germany      Mens      FALSE
#> 6  1985 Oscar Rodriguez    11.8 United States Mens      FALSE
```

Strategy 2: pivot long, separate, pivot wide

Steps:

1. Read in the data
2. Clean the names
3. Remove * note at bottom of table

```
1 hot_dogs <- read_excel(  
2   file.path('data', 'hot_dog_winners.xlsx'),  
3   sheet = 'hot_dog_winners'  
4 ) %>%  
5   clean_names() %>%  
6   dplyr::filter(!is.na(mens))  
7  
8 glimpse(hot_dogs)
```

```
#> Rows: 40  
#> Columns: 7  
#> $ year      <chr> "1980", "1981", "1982", "1983", "1984",  
#> $ mens      <chr> "Paul Siederman & Joe Baldini", "Thomson & Smith",  
#> $ dogs_eaten_3 <chr> "9.1", "11", "11", "19.5", "9.5", "16",  
#> $ country_4   <chr> "United States", "United States", "United States",  
#> $ womens     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA,  
#> $ dogs_eaten_6 <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA,  
#> $ country_7   <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA,
```

Strategy 2: pivot long, separate, pivot wide

Steps:

1. Read in the data
2. Clean the names
3. Remove * note at bottom of table
4. **Rename variables**
5. **Gather all the "joint" variables**

```
1 hot_dogs <- hot_dogs %>%
2   select(
3     year,
4     competitor.mens = mens,
5     competitor.womens = womens,
6     dogs_eaten.mens = dogs_eaten_3,
7     dogs_eaten.womens = dogs_eaten_6,
8     country.mens = country_4,
9     country.womens = country_7
10  ) %>%
11  pivot_longer(
12    names_to = 'variable',
13    values_to = 'value',
14    competitor.mens:country.womens
15  )
16
17 head(hot_dogs, 3)
```

```
#> # A tibble: 3 × 3
#>   year variable      value
#>   <chr> <chr>      <chr>
#> 1 1980 competitor.mens Paul Siederman & Joe Baldini
```

Strategy 2: pivot long, separate, pivot wide

Steps:

1. Read in the data
2. Clean the names
3. Remove * note at bottom of table
4. Rename variables
5. Gather all the "joint" variables
6. **Separate "joint" variables into components**

```
1 hot_dogs <- hot_dogs %>%
2   separate(
3     variable,
4     into = c('variable', 'competition'),
5     sep = '\\\.'
6   )
7
8 head(hot_dogs)
```

```
#> # A tibble: 6 × 4
#>   year variable competition value
#>   <chr> <chr>      <chr>      <chr>
#> 1 1980 competitor mens      Paul Siederman & Joe Baldini
#> 2 1980 competitor womens    <NA>
#> 3 1980 dogs_eaten mens      9.1
#> 4 1980 dogs_eaten womens    <NA>
#> 5 1980 country   mens      United States
#> 6 1980 country   womens    <NA>
```

Strategy 2: pivot long, separate, pivot wide

Steps:

```
1 hot_dogs <- hot_dogs %>%
2   pivot_wider(names_from = variable, values_from = value) %>%
3   mutate(
4     new_record = str_detect(dogs_eaten, "\\*"),
5     dogs_eaten = parse_number(dogs_eaten),
6     year = as.numeric(year)
7   )
8
9 glimpse(hot_dogs)
```

```
#> Rows: 80
#> Columns: 6
#> $ year      <dbl> 1980, 1980, 1981, 1981, 1982, 1982, 1983, 1983
#> $ competition <chr> "mens", "womens", "mens", "womens", "mens", "v
#> $ competitor  <chr> "Paul Siederman & Joe Baldini", NA, "Thomas De
#> $ dogs_eaten  <dbl> 9.10, NA, 11.00, NA, 11.00, NA, 19.50, NA, 9.5
#> $ country     <chr> "United States", NA, "United States", NA, "Un
#> $ new_record  <lgl> FALSE, NA, FALSE, NA, FALSE, NA, FALSE, NA, FA
```

Divide & conquer

```
1 hot_dogs <- read_excel(
2   file.path('data', 'hot_dog_winners.xlsx'),
3   sheet = 'hot_dog_winners'
4 ) %>%
5   clean_names() %>%
6   dplyr::filter(!is.na(mens))
7
8 # Divide
9 hot_dogs_m <- hot_dogs %>%
10  select(
11    year,
12    competitor = mens,
13    dogs_eaten = dogs_eaten_3,
14    country = country_4
15  ) %>%
16  mutate(competition = 'Mens')
17 hot_dogs_w <- hot_dogs %>%
18  select(
19    year,
20    competitor = womens,
21    dogs_eaten = dogs_eaten_6,
22    country = country_7
23  ) %>%
24  mutate(competition = 'Womens') %>%
25  dplyr::filter(!is.na(competitor))
26
27 # Merge and finish cleaning
28 hot_dogs <- bind_rows(hot_dogs_m, hot_dogs_w) %>%
29  mutate(
30    new_record = str_detect(dogs_eaten, "\\*"),
31    dogs_eaten = parse_number(dogs_eaten),
32    year = as.numeric(year)
33  )
```

Pivot long, separate, pivot wide

```
1 hot_dogs <- read_excel(
2   file.path('data', 'hot_dog_winners.xlsx'),
3   sheet = 'hot_dog_winners'
4 ) %>%
5   clean_names() %>%
6   dplyr::filter(!is.na(mens)) %>%
7
8   # Rename variables
9   select(
10    year,
11    competitor.mens = mens,
12    competitor.womens = womens,
13    dogs_eaten.mens = dogs_eaten_3,
14    dogs_eaten.womens = dogs_eaten_6,
15    country.mens = country_4,
16    country.womens = country_7
17  ) %>%
18  # Gather "joint" variables
19  pivot_longer(
20    names_to = 'variable',
21    values_to = 'value',
22    competitor.mens:country.womens
23  ) %>%
24  # Separate "joint" variables
25  separate(variable, into = c('variable', 'competition'), sep = '\\\\.') %>%
26  # Spread "joint" variables
27  pivot_wider(names_from = variable, values_from = value) %>%
28  # Finish cleaning
29  mutate(
30    new_record = str_detect(dogs_eaten, "\\*"),
31    dogs_eaten = parse_number(dogs_eaten),
32    year = as.numeric(year)
33  )
```

Strategies for dealing with sub-headers

Example:

OICA passenger car sales data

	A	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1		NEW PC REGISTRATIONS OR SALES													
2															
3															
4															
5		<i>Estimated figures</i>													
6	REGIONS/COUNTRIES	2005	2006	2007	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018
7															
8	EUROPE	17,906,455	18,685,556	19,618,588	18,821,599	16,608,761	16,499,863	17,167,600	16,191,269	15,942,273	16,154,279	16,410,563	17,291,819	17,974,281	17,912,336
9	EU 28 countries + EFTA	15,622,035	15,961,138	16,147,274	14,911,880	14,533,115	13,830,694	13,642,659	12,567,903	12,344,415	13,061,461	14,287,881	15,160,239	15,631,283	15,626,509
10	EU 15 countries + EFTA	14,565,695	14,820,182	14,842,186	13,602,038	13,668,808	12,984,549	12,815,435	11,773,281	11,555,153	12,148,648	13,261,258	13,971,468	14,320,223	14,210,016
11	AUSTRIA	307,915	308,594	298,182	293,697	319,403	328,563	356,145	336,010	319,035	303,318	308,555	329,604	353,320	341,068
12	BELGIUM	480,088	526,141	524,795	535,947	476,194	547,340	572,211	486,737	486,065	482,939	501,066	539,519	546,558	549,632
13	DENMARK	148,819	156,936	162,686	150,199	112,454	153,858	170,036	170,763	182,086	189,055	207,717	222,924	221,821	218,566
14	FINLAND	148,161	145,700	125,608	139,669	90,574	111,968	126,123	111,251	103,455	106,237	108,819	118,991	120,480	120,480
15	FRANCE	2,118,042	2,045,745	2,109,672	2,091,369	2,302,398	2,251,669	2,204,229	1,898,760	1,790,456	1,795,885	1,917,226	2,015,177	2,110,748	2,173,481
16	GERMANY	3,319,259	3,467,961	3,148,163	3,090,040	3,807,175	2,916,259	3,173,634	3,082,504	2,952,431	3,036,773	3,206,042	3,351,607	3,441,262	3,435,778
17	GREECE	269,728	267,669	279,745	267,295	219,730	141,501	97,680	58,482	58,694	71,218	75,805	78,873	88,083	103,431
18	ICELAND	18,080	17,129	15,942	9,033	2,113	3,106	5,038	7,902	7,274	9,537	14,004	18,442	21,324	17,976
19	IRELAND	171,742	178,484	186,325	151,607	57,453	88,446	89,911	79,498	74,367	96,284	124,804	146,600	131,332	125,557
20	ITALY	2,244,108	2,335,462	2,494,115	2,161,359	2,159,465	1,961,580	1,749,740	1,403,010	1,304,648	1,360,578	1,575,737	1,824,968	1,970,497	1,910,025
21	LUXEMBOURG	48,517	50,837	51,332	52,359	47,265	49,726	49,881	50,398	46,624	49,793	46,473	50,561	52,775	52,786
22	NETHERLANDS	465,196	483,999	504,300	499,980	387,699	482,531	555,812	502,454	417,036	387,553	449,350	382,825	414,306	443,531
23	NORWAY	109,907	109,164	129,195	110,617	98,675	127,754	138,345	137,967	142,151	144,202	150,686	154,603	158,650	147,929
24	PORTUGAL	206,488	194,702	201,816	213,389	161,013	223,464	153,404	95,309	105,921	142,826	178,503	207,345	222,129	228,327
25	SPAIN	1,528,877	1,634,608	1,614,835	1,161,176	952,772	982,015	808,051	699,589	722,689	890,125	1,094,077	1,147,007	1,234,932	1,321,438
26	SWEDEN	274,301	282,766	306,794	253,982	213,408	289,684	304,984	279,899	269,599	303,948	345,108	372,318	379,393	353,729
27	SWITZERLAND (+FL)	266,770	269,421	284,674	288,525	266,018	294,239	318,958	328,139	307,885	301,942	323,783	317,318	311,996	299,135
28	UNITED KINGDOM	2,439,717	2,344,864	2,404,007	2,131,795	1,994,999	2,030,846	1,941,253	2,044,609	2,264,737	2,476,435	2,633,503	2,692,786	2,540,617	2,367,147
29	EUROPE NEW MEMBERS	1,056,340	1,140,956	1,305,088	1,309,842	864,307	846,145	827,224	794,622	789,262	912,813	1,026,623	1,188,771	1,311,060	1,416,493
30	BULGARIA*	25,956	36,455	43,521	45,143	22,869	16,257	19,250	19,419	19,352	20,359	23,500	26,370	33,265	37,506
31	CROATIA	70,541	78,775	82,664	88,265	44,918	38,587	41,561	31,360	27,802	33,962	35,715	44,106	50,769	60,041
32	CYPRUS	17,687	18,639	22,878	22,241	14,981	14,088	13,480	10,123	7,102	8,276	10,344	12,643	13,127	13,135
33	CZECH REPUBLIC	151,699	156,686	174,456	182,554	167,708	169,580	173,595	174,009	164,736	192,314	230,857	259,693	271,595	261,437
34	ESTONIA	19,640	25,363	30,912	24,579	9,946	10,295	17,070	19,424	19,694	20,969	20,347	22,429	25,618	26,297
35	HUNGARY	198,982	187,676	171,661	153,278	60,189	43,476	45,094	53,059	56,139	67,476	77,171	96,552	116,265	136,601
36	LATVIA	10,467	14,234	21,606	22,217	7,515	7,970	13,234	10,665	10,636	12,452	13,765	16,359	16,698	16,878
37	LITHUANIA	16,602	25,582	32,771	19,831	5,367	6,365	10,980	12,165	12,163	14,503	17,085	20,320	25,836	32,382
38	MALTA	6,552	6,745	6,240	5,423	5,894	4,056	5,428	5,884	5,749	6,451	7,121	7,333	7,825	8,128
39	POLAND	207,007	224,728	277,427	319,190	276,220	315,855	277,427	272,719	289,913	327,709	354,975	416,123	486,352	531,889
40	ROMANIA	214,967	247,411	312,533	285,506	116,016	94,441	81,709	66,436	57,710	82,809	98,325	115,004	105,083	129,004
41	SLOVAKIA	56,916	59,084	59,700	70,040	74,717	64,033	68,203	69,268	65,998	72,237	77,968	88,165	96,105	98,080
42	SLOVENIA	59,324	59,578	68,719	71,575	57,967	61,142	60,193	50,091	52,268	53,296	59,450	63,674	62,522	65,115
43	RUSSIA, TURKEY & OTHER EUROPE	2,284,420	2,724,418	3,471,314	3,909,719	2,075,646	2,669,169	3,524,941	3,623,366	3,597,858	3,092,818	2,122,682	2,131,580	2,342,998	2,285,827

Strategies for dealing with sub-headers

Steps:

1. Read in the data, skipping first 5 rows
2. Clean the names

```
1 pc_sales <- read_excel(  
2   file.path('data', 'pc_sales_2018.xlsx'),  
3   sheet = 'pc_sales',  
4   skip = 5  
5 ) %>%  
6   clean_names() %>%  
7   rename(country = regions_countries)  
8  
9 glimpse(pc_sales)
```

```
#> Rows: 160  
#> Columns: 18  
#> $ country <chr> NA, "EUROPE", "EU 28 countries + EFTA",  
#> $ x2      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,  
#> $ x3      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,  
#> $ x4      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,  
#> $ x2005   <dbl> NA, 17906455, 15622035, 14565695, 307915  
#> $ x2006   <dbl> NA, 18685556, 15961138, 14820182, 308594  
#> $ x2007   <dbl> NA, 19618588, 16147274, 14842186, 298182  
#> $ x2008   <dbl> NA, 18821599, 14911880, 13602038, 293697  
#> $ x2009   <dbl> NA, 16608761, 14533115, 13668808, 319403  
#> $ x2010   <dbl> NA, 16499863, 13830694, 12984549, 328563
```

Strategies for dealing with sub-headers

Steps:

1. Read in the data,
skipping first 5 rows
2. Clean the names
3. **Drop bad columns**
4. **Filter out bad rows**

Use **datapasta** to get
rows to drop

```
1 drop <- c(
2   'EUROPE',
3   'EU 28 countries + EFTA',
4   'EU 15 countries + EFTA',
5   'EUROPE NEW MEMBERS',
6   'RUSSIA, TURKEY & OTHER EUROPE',
7   'AMERICA',
8   'NAFTA',
9   'CENTRAL & SOUTH AMERICA',
10  'ASIA/OCEANIA/MIDDLE EAST',
11  'AFRICA',
12  'ALL COUNTRIES'
13 )
14
15 pc_sales <- pc_sales %>%
16   select(-c(x2:x4)) %>% # Drop bad columns
17   filter(
18     !country %in% drop, # Drop bad rows
19     !is.na(country)
20   )
21
22 head(pc_sales)
```

```
#> # A tibble: 6 × 15
#>   country  x2005  x2006  x2007  x2008  x2009  x2010  x2011  x2012  x2013  x2014
#>   <chr>    <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
#> 1 AUSTRIA  307915  308594  298182  293697  319403  328563  356145  336010  319035  303318
#> 2 BELGIUM  480088  526141  524795  535947  476194  547340  572211  486737  486065  482939
#> 3 DENMARK  148819  156936  162686  150199  112454  153858  170036  170763  182086  189055
#> 4 FINLAND  148161  145700  125608  139669   90574  111968  126123  111251  103455  106237
```

Strategies for dealing with sub-headers

Steps:

1. Read in the data, skipping first 5 rows
2. Clean the names
3. Drop bad columns
4. Filter out bad rows
5. **Gather the year variables**

```
1 pc_sales <- pc_sales %>%  
2   pivot_longer(  
3     names_to = 'year',  
4     values_to = 'num_cars',  
5     cols = x2005:x2018  
6   )  
7  
8 head(pc_sales)
```

```
#> # A tibble: 6 × 3  
#>   country year  num_cars  
#>   <chr>   <chr>    <dbl>  
#> 1 AUSTRIA x2005    307915  
#> 2 AUSTRIA x2006    308594  
#> 3 AUSTRIA x2007    298182  
#> 4 AUSTRIA x2008    293697  
#> 5 AUSTRIA x2009    319403  
#> 6 AUSTRIA x2010    328563
```

Strategies for dealing with sub-headers

Steps:

1. Read in the data, skipping first 5 rows
2. Clean the names
3. Drop bad columns
4. Filter out bad rows
5. Gather the year variables
6. **Separate the "x" from the year**

```
1 pc_sales <- pc_sales %>%  
2   separate(  
3     year,  
4     into = c('drop', 'year'),  
5     sep = 'x',  
6     convert = TRUE  
7   )  
8  
9 head(pc_sales)
```

```
#> # A tibble: 6 × 4  
#>   country drop   year num_cars  
#>   <chr>   <lgl> <int>   <dbl>  
#> 1 AUSTRIA NA     2005   307915  
#> 2 AUSTRIA NA     2006   308594  
#> 3 AUSTRIA NA     2007   298182  
#> 4 AUSTRIA NA     2008   293697  
#> 5 AUSTRIA NA     2009   319403  
#> 6 AUSTRIA NA     2010   328563
```

Strategies for dealing with sub-headers

Steps:

```
1 pc_sales <- pc_sales %>%  
2   select(-drop) %>%  
3   mutate(country = str_to_title(country))  
4  
5 head(pc_sales)
```

```
#> # A tibble: 6 × 3  
#>   country year num_cars  
#>   <chr>   <int>   <dbl>  
#> 1 Austria  2005    307915  
#> 2 Austria  2006    308594  
#> 3 Austria  2007    298182  
#> 4 Austria  2008    293697  
#> 5 Austria  2009    319403  
#> 6 Austria  2010    328563
```

What if I wanted to keep the continents?

Strategy: Join a new data frame linking country -> continent

Strategy 1: Find another source

Strategy 2: Hand-make it

```
1 pc_regions <- read_csv(file.path(
2   "data",
3   "pc_regions.csv"
4 ))
5
6 head(pc_regions)
```

```
#> # A tibble: 6 × 3
#>   country region subregion
#>   <chr>    <chr>   <chr>
#> 1 AUSTRIA  EUROPE  EU 15 countries + EFTA
#> 2 BELGIUM  EUROPE  EU 15 countries + EFTA
#> 3 DENMARK  EUROPE  EU 15 countries + EFTA
#> 4 FINLAND  EUROPE  EU 15 countries + EFTA
#> 5 FRANCE   EUROPE  EU 15 countries + EFTA
#> 6 GERMANY  EUROPE  EU 15 countries + EFTA
```

```
1 pc_sales <- pc_sales %>%
2   left_join(pc_regions)
3
4 head(pc_sales)
```

```
#> # A tibble: 6 × 5
#>   country year num_cars region subregion
#>   <chr>   <int>   <dbl> <chr>   <chr>
#> 1 AUSTRIA  2005     307915 EUROPE  EU 15 countries
#> 2 AUSTRIA  2006     308594 EUROPE  EU 15 countries
#> 3 AUSTRIA  2007     298182 EUROPE  EU 15 countries
#> 4 AUSTRIA  2008     293697 EUROPE  EU 15 countries
#> 5 AUSTRIA  2009     319403 EUROPE  EU 15 countries
#> 6 AUSTRIA  2010     328563 EUROPE  EU 15 countries
```

	A	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1															
2															
3															
4															
5															
6															
7															
8															
9															
10															
11															
12															
13															
14															
15															
16															
17															
18															
19															
20															
21															
22															
23															
24															
25															
26															
27															
28															
29															
30															
31															
32															
33															
34															
35															
36															
37															
38															
39															
40															
41															
42															
43															

```

1 drop <- c(
2   'EUROPE',
3   'EU 28 countries + EFTA',
4   'EU 15 countries + EFTA',
5   'EUROPE NEW MEMBERS',
6   'RUSSIA, TURKEY & OTHER EUROPE',
7   'AMERICA',
8   'NAFTA',
9   'CENTRAL & SOUTH AMERICA',
10  'ASIA/OCEANIA/MIDDLE EAST',
11  'AFRICA',
12  'ALL COUNTRIES'
13 )
14
15 pc_regions <- read_csv(file.path("data", "pc_regions.csv"))
16
17 pc_sales <- read_excel(
18   file.path('data', 'pc_sales_2018.xlsx'),
19   sheet = 'pc_sales',
20   skip = 5
21 ) %>%
22   clean_names() %>%
23   rename(country = regions_countries) %>%
24   select(-c(x2:x4)) %>% # Drop bad columns
25   filter(
26     !country %in% drop, # Drop bad rows
27     !is.na(country)
28   ) %>%
29   pivot_longer(
30     names_to = 'year',
31     values_to = 'num_cars',
32     cols = x2005:x2018
33   ) %>%
34   separate(year, into = c('drop', 'year'), sep = 'x', convert = TRUE) %>%

```

```

#> # A tibble: 6 × 5
#>   country year num_cars region subregion
#>   <chr>   <int>   <dbl> <chr>   <chr>
#> 1 Austria  2005     307915 Europe Eu 15 Countries + Efta
#> 2 Austria  2006     308594 Europe Eu 15 Countries + Efta
#> 3 Austria  2007     298182 Europe Eu 15 Countries + Efta
#> 4 Austria  2008     293697 Europe Eu 15 Countries + Efta
#> 5 Austria  2009     319403 Europe Eu 15 Countries + Efta

```